

Infrastructure, packages et exploitation

ÉTUDE DE DOMAINE

Les 34 composants de support : bibliothèques, charts et images

Le retrait des services Kotlin ne suffit pas à produire une plateforme sans JVM. Le contrat porte aussi sur les bibliothèques, générations de clients, runtimes, opérateurs, jobs et outils de secours.

packages : responsabilités et trajectoires

- Les lignes décrivent la disposition proposée de chaque dossier recensé. Les métadonnées du planner et les fichiers d'entrée étayent son rôle et ses dépendances ; elles ne constituent pas un audit ligne par ligne de tout le composant. Les gates ci-dessous restent à exécuter.

Composant	Rôle	Trajectoire proposée	Gate de retrait ou maintien
packages/cleyrop-common	Utilitaires Python communs	Conserver les adaptateurs utilisateurs ; transférer politique et invariants dans des crates Rust partagées.	Parité des erreurs, identifiants et contexte tenant.
packages/cleyrop-sdk	SDK de développement	Préserver Python/SQL et ajouter source(), manifests, pins, run et receipts.	Notebook existant migré sans écriture directe sur main.
packages/cleyrop-spring-auth	Authentification Spring et intégration FGA	Retirer la bibliothèque JVM ; extraire contrats et fixtures vers le middleware Rust.	Même décision sur une matrice de sujets/actions/ressources.
packages/cleyrop-testing	Fixtures et aides Python	Réutiliser les jeux de contrat entre anciens et nouveaux services.	Tests clients identiques contre API de compatibilité puis Rust.
packages/client-python	Client de plateforme	Générer un client versionné avec types de pins, erreurs et opérations asynchrones.	Pagination, cancellation et retries ne dupliquent pas les effets.
packages/downloads-site	Distribution des clients	Conserver le site et signer les distributions des clients migrés.	Liens, checksums et compatibilité des versions contrôlés.
packages/error-lib	Contrat d'erreurs JVM	Retirer le JAR ; garder codes stables et corrélation dans Rust et clients.	Erreurs machine stables, aucun détail de ressource interdite.
packages/keycloak-plugins	Extensions Keycloak	Reproduire les comportements nécessaires via IdP natif et IAM Rust.	Recette de fédération, claims, sessions et révocation.
packages/keycloak-theme	Écrans et packaging Keycloak	Porter les écrans vers l'IdP retenu, supprimer le JAR.	Connexion, récupération, MFA, accessibilité et langues.
packages/name-lib	Conventions de nommage JVM	Porter les règles utiles dans une crate sans dépendance JVM.	Corpus de noms existants et collisions conservé.

Composant	Rôle	Trajectoire proposée	Gate de retrait ou maintien
packages/proto	Contrats protobuf et génération	Préserver les contrats nécessaires ; remplacer Gradle par génération sans JVM.	Wire compatibility, champs réservés et clients mixtes.
packages/pyaicleyrop	Bibliothèque IA Python	Conserver côté utilisateur ; médiation des appels par core Rust et proxy gouverné.	Identité, quotas, isolation et provenance des artefacts.

Références de l'analyse

- [monorepo · packages/cleyrop-common/README.md:1](#) Point d'entrée et inventaire du composant packages/cleyrop-common
- [monorepo · packages/cleyrop-sdk/README.md:1](#) Point d'entrée et inventaire du composant packages/cleyrop-sdk
- [monorepo · packages/cleyrop-spring-auth/README.md:1](#) Point d'entrée et inventaire du composant packages/cleyrop-spring-auth
- [monorepo · packages/cleyrop-testing/README.md:1](#) Point d'entrée et inventaire du composant packages/cleyrop-testing
- [monorepo · packages/client-python/README.md:1](#) Point d'entrée et inventaire du composant packages/client-python
- [monorepo · packages/downloads-site/README.md:1](#) Point d'entrée et inventaire du composant packages/downloads-site
- [monorepo · packages/error-lib/README.md:1](#) Point d'entrée et inventaire du composant packages/error-lib
- [monorepo · packages/keycloak-plugins/README.md:1](#) Point d'entrée et inventaire du composant packages/keycloak-plugins
- [monorepo · packages/keycloak-theme/Dockerfile:1](#) Point d'entrée et inventaire du composant packages/keycloak-theme
- [monorepo · packages/name-lib/README.md:1](#) Point d'entrée et inventaire du composant packages/name-lib
- [monorepo · packages/proto/README.md:1](#) Point d'entrée et inventaire du composant packages/proto
- [monorepo · packages/pyaicleyrop/README.md:1](#) Point d'entrée et inventaire du composant packages/pyaicleyrop

charts : responsabilités et trajectoires

- Les lignes décrivent la disposition proposée de chaque dossier recensé. Les métadonnées du planner et les fichiers d'entrée étayent son rôle et ses dépendances ; elles ne constituent pas un audit ligne par ligne de tout le composant. Les gates ci-dessous restent à exécuter.

Composant	Rôle	Trajectoire proposée	Gate de retrait ou maintien
charts/cleyrop-backup	Sauvegarde plateforme	Étendre le manifeste de restauration à toutes les autorités et journaux.	DR-1 à DR-8, aucun effet relancé par restauration.
charts/cleyrop-common	Bibliothèque Helm	Réutiliser avec profils contrôle/données, capacités et service accounts séparés.	Lint et rendu : pas de secret partagé ni JVM transitive.
charts/cnpg-operator	Opérateur PostgreSQL	Conserver et qualifier sauvegarde/restauration des autorités de contrôle.	PITR réel, bascule et compatibilité des extensions.
charts/gitea	Versionnement des projets	Conserver pour code et drafts, distinguer branches Git et éditions de données.	Webhook rejoué une seule fois, accès projet et restauration.
charts/jupyterhub	Sessions de développement	Conserver le service utilisateur après changement des images et credentials.	Notebook sans JRE, lecture épinglée et écriture via Flows.

Composant	Rôle	Trajectoire proposée	Gate de retrait ou maintien
charts/kafka-operator	Déploiement du journal Kafka	Retirer Kafka/Strimzi ; qualifier un journal durable sans JVM.	Offsets, replay, rétention, panne disque/nœud et protocoles clients.
charts/keycloak-operator	Cycle de vie Keycloak	Retirer après migration de l'IdP et de toutes les fédérations.	Plus aucun realm, SPI ou initContainer JVM résiduel.
charts/postgres	Clusters PostgreSQL	Conserver les autorités nécessaires ; différencier caches restructurables et contrôle durable.	Chaîne WAL réellement restaurable et restauration coordonnée.
charts/registry	Distribution des images	Conserver, signer et inventorier les images finales par digest.	Images applicatives, jobs, hooks et initContainers sans JRE/JAR.
charts/sftp	Entrée fichiers SFTP	Conserver l'entrée protocolaire ; publier des FILESET après intégrité et scan.	Upload incomplet invisible ; doublon et retry sans double édition.
charts/trino	SQL distribué JVM	Retirer au terme de la qualification workload de Serving Rust et du moteur distribué.	Parité types, SQL, ACL, charge et annulation sur corpus client.
charts/valkey	Cache et échanges rapides	Conserver pour données dérivées ; aucune autorité de checkpoint source.	Perte totale du cache sans perte d'édition ni effet dupliqué.

Références de l'analyse

- [monorepo · charts/cleyrop-backup/README.md:1](#) Point d'entrée et inventaire du composant charts/cleyrop-backup
- [monorepo · charts/cleyrop-common/Chart.yaml:1](#) Point d'entrée et inventaire du composant charts/cleyrop-common
- [monorepo · charts/cnpg-operator/AGENTS.md:1](#) Point d'entrée et inventaire du composant charts/cnpg-operator
- [monorepo · charts/gitea/AGENTS.md:1](#) Point d'entrée et inventaire du composant charts/gitea
- [monorepo · charts/jupyterhub/Dockerfile:1](#) Point d'entrée et inventaire du composant charts/jupyterhub
- [monorepo · charts/kafka-operator/README.md:1](#) Point d'entrée et inventaire du composant charts/kafka-operator
- [monorepo · charts/keycloak-operator/README.md:1](#) Point d'entrée et inventaire du composant charts/keycloak-operator
- [monorepo · charts/postgres/AGENTS.md:1](#) Point d'entrée et inventaire du composant charts/postgres
- [monorepo · charts/registry/Chart.yaml:1](#) Point d'entrée et inventaire du composant charts/registry
- [monorepo · charts/sftp/AGENTS.md:1](#) Point d'entrée et inventaire du composant charts/sftp
- [monorepo · charts/trino/README.md:1](#) Point d'entrée et inventaire du composant charts/trino
- [monorepo · charts/valkey/AGENTS.md:1](#) Point d'entrée et inventaire du composant charts/valkey

hub : responsabilités et trajectoires

- Les lignes décrivent la disposition proposée de chaque dossier recensé. Les métadonnées du planner et les fichiers d'entrée étayent son rôle et ses dépendances ; elles ne constituent pas un audit ligne par ligne de tout le composant. Les gates ci-dessous restent à exécuter.

Composant	Rôle	Trajectoire proposée	Gate de retrait ou maintien
hub/ci	Construction des runtimes	Adapter matrice, SBOM et tests aux images sans JVM.	Aucune étape Gradle/Maven/java requise dans la chaîne finale.
hub/e2e	Acceptation des runtimes	Remplacer les attentes Spark par scénarios SDK/Flight/SQL qualifiés.	Même jeu utilisateur exécuté dans chaque image publiée.
hub/jupyterlab	Image notebook incluant Spark/Java	Reconstruire avec Python, Arrow et SDK ; moteurs via contrats explicites.	Import, dataframes, dépendances et kernels sans JRE.
hub/python	Image Python utilisateur	Conserver langage et isolation ; injecter seulement des capacités limitées.	Exécution bornée, secrets, egress et reprise contrôlés.
hub/shared	Ressources communes des runtimes	Revoir bootstrap, configuration et dépendances partagées.	Aucun chemin réintroduisant Spark, JAR ou droits généraux.
hub/spark	Runtime Spark	Retirer ; migrer les notebooks et jobs par corpus, pas par renommage d'image.	SQL, UDF, gros joins, skew, mémoire et délais mesurés.
hub/spark-base	Base Java/Spark	Retirer après extinction de toutes les images filles.	Graphe de dépendances et SBOM sans descendants JVM.

Références de l'analyse

- [monorepo](#) · [ci/components.json:1](#) Point d'entrée et inventaire du composant hub/ci
- [monorepo](#) · [ci/components.json:1](#) Point d'entrée et inventaire du composant hub/e2e
- [monorepo](#) · [hub/jupyterlab/Dockerfile:1](#) Point d'entrée et inventaire du composant hub/jupyterlab
- [monorepo](#) · [hub/python/Dockerfile:1](#) Point d'entrée et inventaire du composant hub/python
- [monorepo](#) · [ci/components.json:1](#) Point d'entrée et inventaire du composant hub/shared
- [monorepo](#) · [hub/spark/Dockerfile:1](#) Point d'entrée et inventaire du composant hub/spark
- [monorepo](#) · [hub/spark-base/Dockerfile:1](#) Point d'entrée et inventaire du composant hub/spark-base

images : responsabilités et trajectoires

- Les lignes décrivent la disposition proposée de chaque dossier recensé. Les métadonnées du planner et les fichiers d'entrée étayent son rôle et ses dépendances ; elles ne constituent pas un audit ligne par ligne de tout le composant. Les gates ci-dessous restent à exécuter.

Composant	Rôle	Trajectoire proposée	Gate de retrait ou maintien
images/alpine-tools	Outils opérationnels	Conserver après inventaire des scripts et capacités.	Jobs d'installation, secours et maintenance sans JVM.
images/cleyrop-devspace	Environnement de développement	Adapter aux clients natifs, génération protobuf et tests de contrat.	Parcours développeur complet sans Java.
images/trivy-with-curl	Image de contrôle sécurité	Conserver et intégrer les contrôles de composition des images.	Rapport par digest et politique de traitement des écarts.

Références de l'analyse

[monorepo](#) · [images/alpine-tools/Dockerfile:1](#) Point d'entrée et inventaire du composant images/alpine-tools

[monorepo](#) · [images/cleyrop-devspace/Dockerfile:1](#) Point d'entrée et inventaire du composant images/cleyrop-devspace

[monorepo](#) · [images/trivy-with-curl/Dockerfile:1](#) Point d'entrée et inventaire du composant images/trivy-with-curl

ÉTUDE DE DOMAINE

Exploitation, restauration et retrait progressif des JVM

La migration avance par cohortes de workloads acceptés. Une durée globale ou un niveau de performance ne sont pas chiffrés sans volumétrie, corpus client et mesure.

Ordre de migration et critères de sortie

Lot	Périmètre	Preuve exigée
M0 · Contrats	Inventaire, compatibilité API, jeux de données et autorité des états	Toutes les capacités de la matrice ont un propriétaire et une recette priorisée.
M1 · Confiance	IAM/PEP, contexte, staging privé, writer/publisher et journal	Veto, fuite staging, zombie/failover et crash publish/ack testés.
M2 · Lecture	Catalogue, Icegres read-only, front/SDK et Eidos projections	Pins stables, découverte filtrée, golden SQL et révocation.
M3 · Ingestion	Une cohorte de connecteurs et FILESET	Bootstrap, incrémental, delete, replay et restauration validés.
M4 · Transformation	Flows, data-dev/process et mutations Eidos	Runs bornés, audit, release multi-sorties et équivalence workload.
M5 · Usages	BI, apps, devspaces, assistant, MCP, ML et exports	Parcours utilisateurs complets et effets externes idempotents.
M6 · Extinction	IdP natif, journal natif, retrait Spark/Trino et chaînes JVM	Inventaire images/processus/jobs/CI sans JVM, restore et exploitation acceptés.

Références de l'analyse

[study](#) · [docs/hemera-v2/08-plateforme-rust-sans-jvm.html:71](#) Périmètre du retrait JVM et gates associés.

Durabilité et reprise

- Le manifeste de sauvegarde coordonne catalogue, objets, UUID/éditions, politique OpenFGA, IAM, clés, drafts, commandes, annulations et reçus d'effets. Une date identique sur plusieurs backups n'établit pas leur compatibilité. Avant réouverture, le plan de données vérifie effectivement manifests, data files, delete files et artefacts.
- Le chart PostgreSQL inspecté prévoit des snapshots quotidiens et l'archivage WAL désactivé par défaut ; le profil micro désactive les sauvegardes. Une configuration n'est pas une preuve de RPO. La rétention doit calculer l'âge réel du point restaurable le plus ancien, en tenant compte des sauvegardes manquées.
- Un rollback de lecture ne déplace pas le checkpoint actif. Restaurer avant des acks irréversibles exige le journal couvrant l'écart. On ne peut ni supposer les messages encore présents ni copier un curseur avancé sur des données anciennes. Les seuils RPO/RTO doivent être contractés par profil puis mesurés.
- Les métriques suivent délai capture/publication/ack, lag des projections, échecs d'audit, conflits, zombies refusés, backlog, quotas, coût par workload et couverture de rétention. Les logs corrént opération/tentative/édition sans contenir secrets ou données sensibles. Le diagnostic distingue timeout, veto, conflit, ack incertain et incident de politique.

Références de l'analyse

- [monorepo · charts/postgres/values.yaml:245](#) Paramètres de sauvegarde du chart inspecté.
- [monorepo · charts/postgres/templates/retention-cronjob.yaml:103](#) Logique de rétention à qualifier.
- [icegres · icegres/src/overwrite.rs:1196](#) Racines d'expiration actuelles à étendre.

Huit campagnes de restauration à rendre exécutables

- Ces scénarios sont des critères proposés, pas des résultats annoncés. Ils complètent I1–I12 et les tests propres à chaque fonctionnalité.

Campagne	Injection	Résultat attendu
DR-1	Perdre uniquement les projections	Reconstruire éditions et statuts sans nouvelle publication ni effet.
DR-2	Perdre la réponse de publication avant ack	Retrouver le reçu exact, puis réconcilier l'ack sans doublon.
DR-3	Restaurer le catalogue derrière les acks	Prouver la couverture du journal, sinon bloquer et rebootstrap sous nouvelle époque.
DR-4	Manquer plusieurs sauvegardes puis lancer la rétention	Conserver toute chaîne WAL nécessaire à un point encore annoncé restaurable.
DR-5	Restaurer IAM et données à des points différents	Rester fermé tant que cohérence, révocations et suppressions ne sont pas réconciliées.
DR-6	Basculer avec une ancienne tentative vivante	Refuser la publication zombie au point de commit, même si la tête n'a pas changé.
DR-7	Interrompre une release de plusieurs sorties	Lecteurs sur ancienne ou nouvelle release complète ; checkpoint commun.
DR-8	Purger avec lecteurs, staging et backups actifs	Aucun objet requis supprimé ; journal de purge repris sans perdre ses racines.

Références de l'analyse

- [study · docs/hemera-v2/08-plateforme-rust-sans-jvm.html:223](#) Résilience et migration de la cible.